UniControl - Spec do Sistema

UniControl — Sistema de Controle de Estoque de Uniformes

1. Objetivo

O UniControl é um sistema web para controlar o estoque de uniformes de empresas que precisam gerenciar a compra, a guarda e a entrega de uniformes aos seus funcionários. Ele resolve o problema de controle feito hoje em planilhas ou no papel, centralizando cadastro, estoque, movimentações e entregas em um só lugar, com suporte a múltiplas empresas clientes na mesma plataforma.

2. Usuários

- **Administrador**: cadastra empresas clientes, cadastra e gerencia usuários do sistema, e tem acesso a todas as funcionalidades do operador.
- **Operador**: cadastra funcionários, cadastra uniformes e suas variações (tamanho/cor), registra entradas/saídas de estoque, registra entregas de uniforme a funcionários e consulta o dashboard e os alertas de estoque baixo.

3. Casos de uso principais

- Usuário faz login no sistema com e-mail e senha.
- Sistema restringe o acesso às telas de Empresas e Usuários apenas para quem tem nível de acesso administrador.
- Administrador cadastra uma nova empresa cliente.
- Administrador cadastra um novo usuário e define seu nível de acesso.
- Operador cadastra um funcionário da empresa.
- Operador cadastra um uniforme (peça) e suas variações de tamanho/cor.
- Operador registra uma entrada de estoque (compra/reposição).
- Operador registra a entrega de um uniforme a um funcionário.
- Operador registra uma saída, devolução ou baixa de estoque.
- Sistema exibe alerta quando o estoque de uma variação atinge o mínimo.
- Usuário consulta o dashboard com o resumo de peças, funcionários, estoque baixo e entregas do mês.

4. Requisitos funcionais

- RF01: O sistema deve permitir login com e-mail e senha.
- RF02: O sistema deve restringir telas de Empresas e Usuários apenas ao nível de acesso administrador.
- RF03: O sistema deve permitir cadastrar empresas clientes (multiempresa).
- RF04: O sistema deve permitir cadastrar funcionários vinculados a uma empresa.
- RF05: O sistema deve permitir cadastrar uniformes (peças) com categoria e descrição.
- RF06: O sistema deve permitir cadastrar variações de um uniforme (tamanho, cor, SKU e estoque mínimo).
- RF07: O sistema deve manter a quantidade em estoque atualizada a cada movimentação (entrada, saída, devolução, baixa).
- RF08: O sistema deve permitir registrar a entrega de uma variação de uniforme a um funcionário, abatendo do estoque automaticamente.
- RF09: O sistema deve impedir o registro de uma entrega ou saída maior que a quantidade disponível em estoque.
- RF10: O sistema deve exibir no dashboard a quantidade total de peças, o número de funcionários ativos, os itens com estoque abaixo do mínimo e o total de entregas do mês.
- RF11: O sistema deve listar, na tela de estoque, todas as variações com destaque visual para as que estão em estoque baixo.

5. Requisitos não funcionais

- Segurança: senhas armazenadas sempre com hash (nunca em texto puro).
- Segurança: toda consulta ao banco de dados feita com parâmetros/prepared statements, nunca concatenando dado do usuário na string SQL.
- Segurança: todo formulário validado também no servidor, não apenas no navegador.
- Segurança: acesso às telas administrativas restrito por nível de acesso, verificado no servidor (não só ocultando o link na tela).
- Segurança: nenhuma senha ou credencial de banco commitada no repositório — dados de conexão isolados em

arquivo de configuração/variável de ambiente.

- Desempenho: cada página deve carregar consultando apenas os dados da empresa do usuário logado (evita retornar dados de outras empresas).

6. Modelo de dados (DER simples)

- **empresas** (1) → (N) **usuarios**: cada usuário pertence a uma empresa.
- **empresas** (1) → (N) **funcionarios**: cada funcionário pertence a uma empresa.
- **empresas** (1) → (N) **uniformes**: cada peça do catálogo pertence a uma empresa.
- **uniformes** (1) → (N) **uniforme_variacoes**: cada peça tem várias variações de tamanho/cor.
- **uniforme_variacoes** (1) → (1) **estoque**: cada variação tem um registro de quantidade atual.
- **uniforme_variacoes** (1) → (N) **movimentacoes**: histórico de entradas/saídas/devoluções/baixas daquela variação.
- **funcionarios** (N) ↔ (N) **uniforme_variacoes**, resolvida pela tabela **entregas**: registra qual funcionário recebeu qual variação, quando e quantas peças.

7. Critérios de aceite

- Dado que um usuário não está logado, quando tenta acessar `dashboard.php` diretamente pela URL, então é redirecionado para a tela de login.
- Dado que um usuário com nível “operador” está logado, quando tenta acessar `empresas.php` ou `usuarios.php`, então o acesso é negado.
- Dado que um operador preenche nome, matrícula, setor e cargo de um novo funcionário e confirma o cadastro, quando a operação é concluída, então o funcionário passa a aparecer na lista de funcionários da empresa.
- Dado que um operador cadastra uma peça de uniforme com nome e categoria, e em seguida adiciona uma variação com tamanho, cor e estoque mínimo, quando a operação é concluída, então a variação aparece disponível para movimentações e entregas, com o estoque inicial informado.
- Dado que uma variação de uniforme tem 3 unidades em estoque, quando um operador tenta registrar uma entrega de 5 unidades, então o sistema bloqueia a operação e informa a quantidade disponível.
- Dado que uma variação de uniforme está com quantidade igual ou menor que o estoque mínimo cadastrado, quando o dashboard é carregado, então essa variação aparece na lista de alertas de estoque baixo.
- Dado que um operador registra uma entrada de 10 unidades de uma variação, quando a operação é concluída, então a quantidade em estoque daquela variação aumenta em 10 e a movimentação aparece no histórico.
- Dado que um administrador cadastra uma nova empresa, quando o cadastro é salvo, então essa empresa passa a estar disponível para vincular novos usuários e funcionários.
- Dado que existem peças em estoque, funcionários ativos, itens com estoque baixo e entregas registradas no mês, quando o dashboard é carregado, então os quatro totais exibidos correspondem exatamente aos dados cadastrados.

8. Decisão de stack e deploy

- **Stack**: PHP (com PDO) + MySQL, sem framework, HTML/CSS próprios.
- **Motivo da escolha**: stack já dominada em sala de aula, com boa compatibilidade com o MySQL Workbench usado na disciplina, e suficiente para o volume de dados do projeto.
- **Deploy**: como o Vercel não executa PHP nativamente, o Vercel fica fora do caminho de deploy final deste projeto. O GitHub continua sendo usado normalmente para versionamento (histórico de commits desde a Fase 2). O sistema será hospedado em um provedor compatível com PHP + MySQL (ex.: InfinityFree, Hostinger ou ambiente local do laboratório, a confirmar com o professor).